

Work Sheet 1 (Data Structures)

1. **Draw** the memory block against each statement in following program. **Identify** the dangling pointers if any.

```
int globalVar = 5;
const int constVar = 10;
void function() {

    int localVar = 20;
    int* heapVar = new int(30);
    *heapVar = 35;
    int* alias = heapVar;
    delete heapVar;
    heapVar = null;
    *alias = 40;
}

int main() {
    // Function Call
    function();
    return 0;
}
```

type	Addr	Memory
text		
data		
bss		
heap		
stack		

2. **Draw** the memory block against each statement in following program. **Identify** memory leaking if any.

<pre> int main() { int rows = 2, cols = 2; int** matrix = new int*[rows]; for (int i = 0; i < rows; i++) { matrix[i] = new int[cols]; } int value = 1; for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) { matrix[i][j] = value++; } } cout << "Matrix: " << endl; for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) { cout << matrix[i][j] << " "; } cout << endl; } // Freeing the allocated memory delete[] matrix; for (int i = 0; i < rows; i++) { delete[] matrix[i]; } return 0; } </pre>			
	type	Addr	Memory
	text		
	data		
	bss		
	heap		
	stack		

3. Problem: Inventory Management for a Point of Sale (POS) System

In a Point of Sale (POS) system for a retail store, you need to manage a dynamic list of products, where the number of products may vary depending on stock. You need to track information like product name, price, and stock quantity. The system should allow the following functionalities:

1. **Add a new product:** Dynamically allocate memory for storing new products.
2. **Sell a product:** When a product is sold, the system should update the stock quantity. If stock becomes zero, the product should be removed from the list.
3. **Display inventory:** Show the list of products currently in stock with their names, prices, and quantities.
4. **Free allocated memory:** Ensure that all dynamically allocated memory is properly freed when the program terminates.

```
// Structure to represent a product
struct Product {
    string name;
    float price;
    int quantity;
};

// Function to add a product to the inventory
void addProduct(Product* &inventory, int &size, string name, float price, int
quantity) {
}

// Function to sell a product
void sellProduct(Product* &inventory, int &size, string name) {

}

// Function to display the inventory
void displayInventory(Product* inventory, int size) {
}

int main() {
    Product* inventory = nullptr;
    int size = 0;

    // Adding products
    addProduct(inventory, size, "Laptop", 999.99, 10);
    addProduct(inventory, size, "Smartphone", 599.99, 5);
```

```

addProduct(inventory, size, "Headphones", 49.99, 20);

// Displaying the inventory
displayInventory(inventory, size);

// Selling a product
sellProduct(inventory, size, "Smartphone");
sellProduct(inventory, size, "Smartphone");

// Displaying the updated inventory
displayInventory(inventory, size);

// Free dynamically allocated memory
delete[] inventory;

return 0;
}

```

You are required to implement the function to manage the inventory.

Explanation of the Program:

- **Dynamic Array Allocation:**
 - The inventory is dynamically managed using pointers. Each time a product is added, a larger array is created, and the contents of the old array are copied into it. Then, the old array is deleted to free memory.
- **Adding a Product:**
 - The function `addProduct()` takes a reference to the current inventory, the current size, and the details of the new product to be added. A new array is allocated with the new size, and the product is appended.
- **Selling a Product:**
 - The `sellProduct()` function reduces the quantity of a product. If the quantity reaches 0, the product is removed from the inventory, and a smaller array is created without that product.
- **Display Function:**
 - The `displayInventory()` function simply loops through the inventory and prints the name, price, and quantity of each product.

4.

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max from each sliding window*.

Note: Use dynamic arrays

Example 1:

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

Window position	Max
-----------------	-----

-----	-----
-------	-------

<code>[1 3 -1]</code>	<code>-3 5 3 6 7</code>	<code>3</code>
-----------------------	-------------------------	----------------

<code>1 [3 -1 -3]</code>	<code>5 3 6 7</code>	<code>3</code>
--------------------------	----------------------	----------------

<code>1 3 [-1 -3 5]</code>	<code>3 6 7</code>	<code>5</code>
----------------------------	--------------------	----------------

<code>1 3 -1 [-3 5 3]</code>	<code>6 7</code>	<code>5</code>
------------------------------	------------------	----------------

<code>1 3 -1 -3 [5 3 6]</code>	<code>7</code>	<code>6</code>
--------------------------------	----------------	----------------

<code>1 3 -1 -3 5 [3 6 7]</code>		<code>7</code>
----------------------------------	--	----------------

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $1 \leq k \leq \text{nums.length}$